

Caesar-LLM experiment

Автор: Хамитов Хаким, tg: @Khakim_Kh

Предыстория и идея:

Еще осенью 2026 года я познакомился с курсом [Zero-to-Hero от Andrej Karpathy](#), о котором хорошо отзывались мои знакомые. На тот момент я почти ничего не знал про Deep Learning, но у меня было представление о нем и была какая-то база в классическом ML. Я начал быстро смотреть видео одно за другим, иногда даже пытался писать какой-то код, так по итогу и дошел до написания трансформера. На этом моменте я выпал. На это были разные причины (возможно мотивация пропала, а возможно учеба – коллоквиум по линалу;)), но ко мне в голову тогда пришла такая мысль: что если обучить LLM на зашифрованном языке? Пока звучит сыровато, но потом мысль начала конкретизироваться, в какой-то степени упрощаться и финальная идея звучала так: что если обучить LLM на тексте, зашифрованном шифром Цезаря со сдвигом 1, сгенерировать с помощью нее текст и попробовать раскодировать обратно? На тот момент для меня это было чем-то неочевидным и любопытным.

В июле 2026 года я решил заново в деталях разобраться в этом курсе (в этот раз намного больше кодил и сам ставил эксперименты) и на этот раз разобрался с трансформерами и сам обучил nanoGPT для генерации текстов в стиле Шекспира. Идея про зашифрованную генерацию с самой осени лежала у меня в заметках и никуда не делась, поэтому пришло время ее реализовать и проверить. Но добавились новые мысли и опять же, больше конкретики:

1) Насколько быстро можно зафайнтюнить модель, знающую обычный язык, на зашифрованную шифром Цезаря версию языка. Ответ на первый взгляд может выглядеть очевидным (понятно, что файнтюнинг скорее должен быть быстрее чем обучение с нуля), но интересно именно численное значение.

2) Как поменяются при этом веса модели? Гипотеза: таблица эмбедингов для букв «сдвинется на один».

Мне кажется это очень упрощенная версия исследования, которое могло бы по итогу быть, но нынешняя вариация выглядит просто и понятно, для таких же новичков как я.

История: За завтраком я задал маме вопрос: “Представь, что тебе нужно научиться говорить на русском языке, только зашифрованным шифром Цезаря. Как думаешь, за какое время ты начала бы на нем свободно коммуницировать?” В качестве ответа была получена величина ~2 месяца, что например, в сравнении с изучением кардинально нового языка – очень маленький промежуток времени (нейронка говорит, что для изучения английского языка до уровня C1 нужно ~700+ часов и переводит это значение в 2-3 года, понятно что спать и есть тоже нужно). И да, тут есть проблемы с тем, что гласные будут превращаться в согласные и скорее всего некоторые слова не получится нормально произносить, но и сам вопрос не сказать что гениальный, так что опустим этот момент.

Обучение и файнтюнинг

Итак, перейдем к делу. Во-первых был принят ряд упрощений:

а) Токенизация посимвольная :)

б) Датасет для обучения – [Tiny Shakespeare](#)

в) Модель не на **124M** параметров, а на **3.66M**, чтобы без мороки запустить на [Kaggle](#) и не ждать слишком долго (терпения видимо у меня совсем мало).

г) Шифром Цезаря сдвигаем только буквы, потому что почему бы и нет.

Сначала я написал классическую версию **GPT**, можно посмотреть на картинке как она выглядела (Рис. 1). Гиперпараметры были выбраны на глаз (почти)! (Рис. 2)

Обучалась вся эта история на платформе Kaggle на **GPU T4**, по времени заняло около получаса на ~**5500** итерациях с early stopping’ом. Лучший лосс на валидационной выборке составил ~**1.516**. График обучения для особо интересующихся тоже показываю (Рис. 3).

```
embed_size = 192
num_head = 4
head_size = embed_size // num_head
num_blocks = 8
batch_size = 64
block_size = 384
```

Рис. 2 Гиперпараметры модели

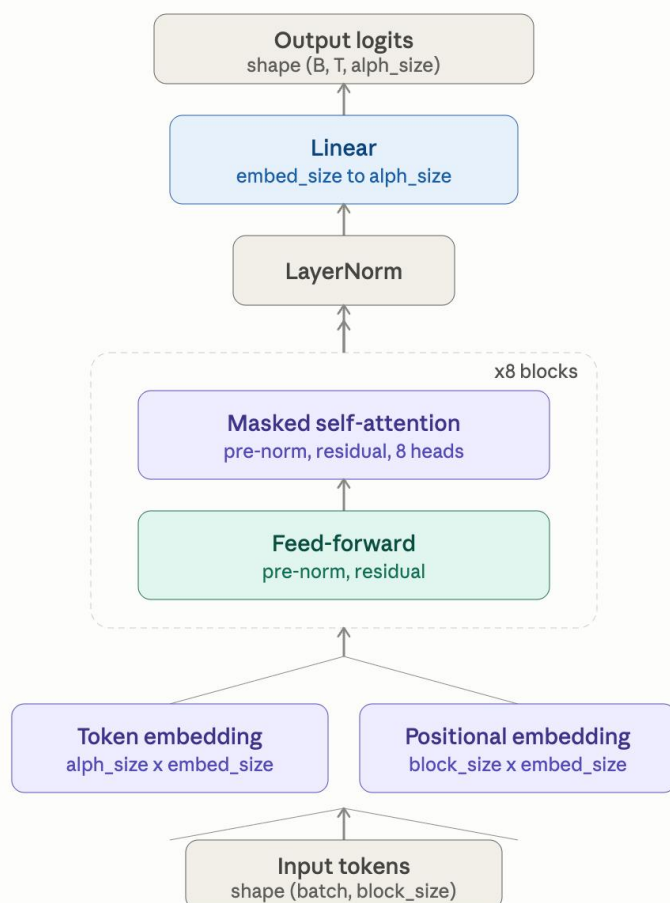


Рис. 1 Модель, которая использовалась для данного исследования. Basically GPT, но меньше по размеру и только для генерации.

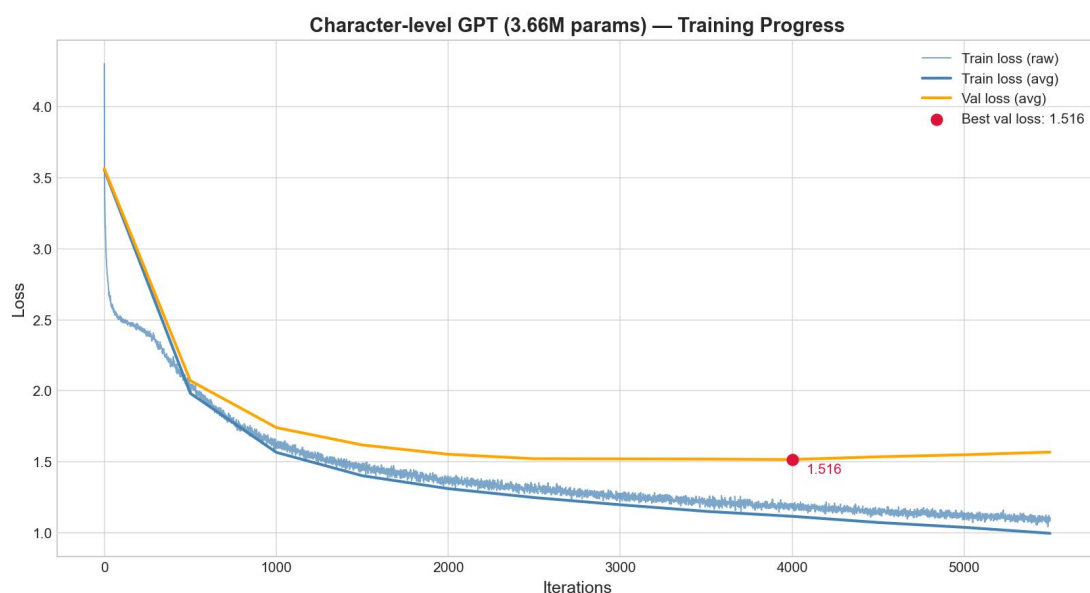


Рис. 3 График обучения классической версии модели.

Дальше я взял лучшую по лоссу на валидационной выборке модель и зафайнтюнил уже на зашифрованном тексте. График обучения тоже прилагаю (Рис. 4). Тут начинается самое интересное.

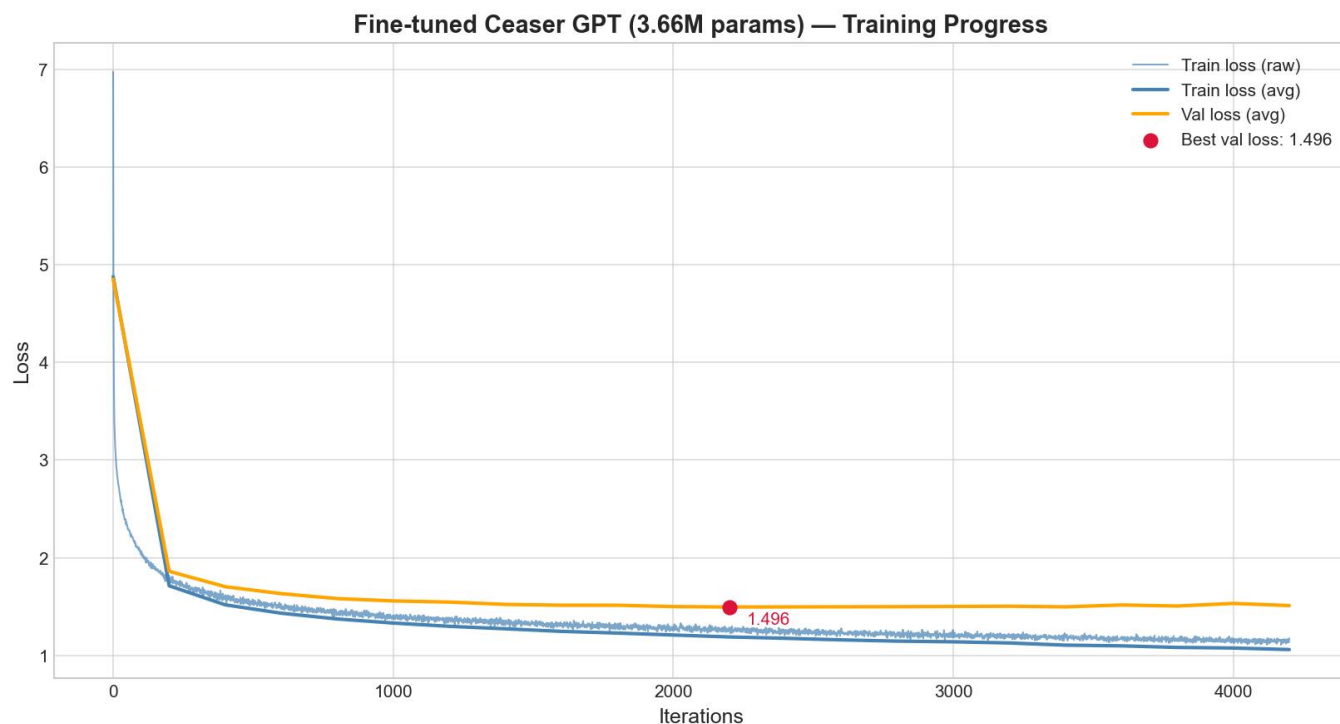


Рис. 4 График обучения Caesar-LLM.

Проверка результатов

Сначала рассмотрим и сравним графики обучения (Рис. 5, 6). Видно, что дообучение достигает лосса **~1.516** в **~2.5** раза быстрее, чем модель с нуля. Видно также, что лосс на валидационной выборке улучшился (**~1.496**), хотя и незначительно, скорее случайность.

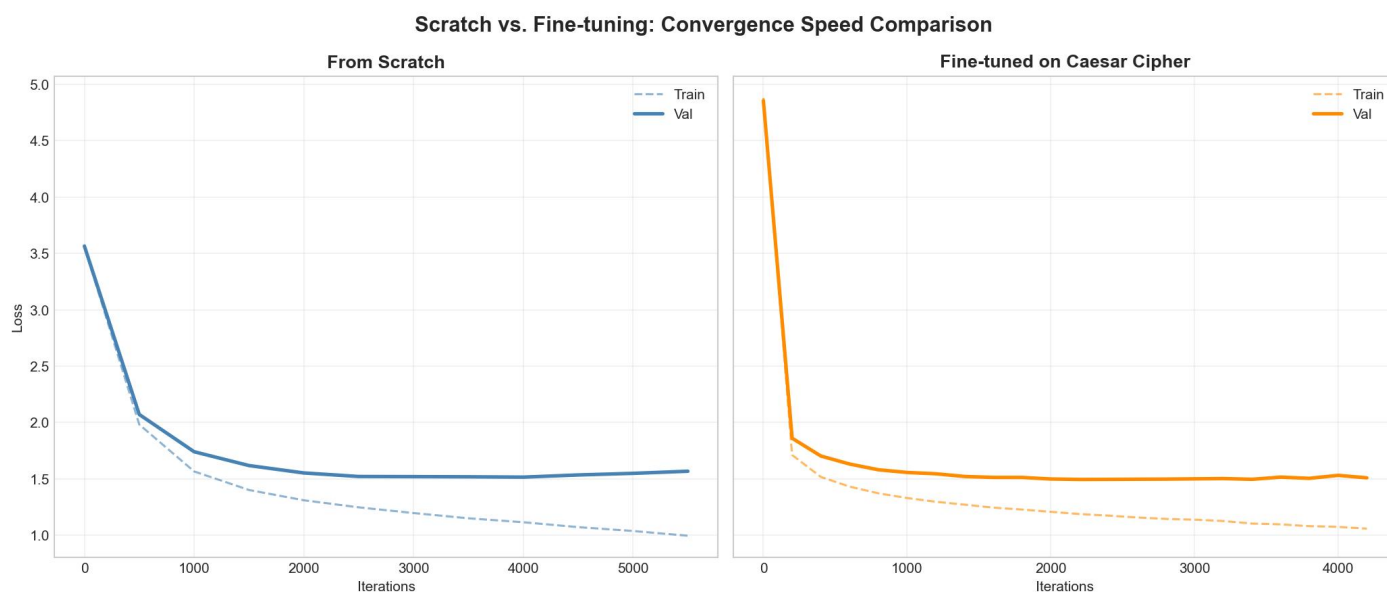


Рис. 5 Сравнение скорости обучения двух версий моделей.

Теперь проверим веса модели. Самое интересно было посмотреть на таблицу эмбедингов для каждого символа, напомним, что изначальная гипотеза заключалась в том, что таблица «съедет» на один по циклу для каждой буквы.

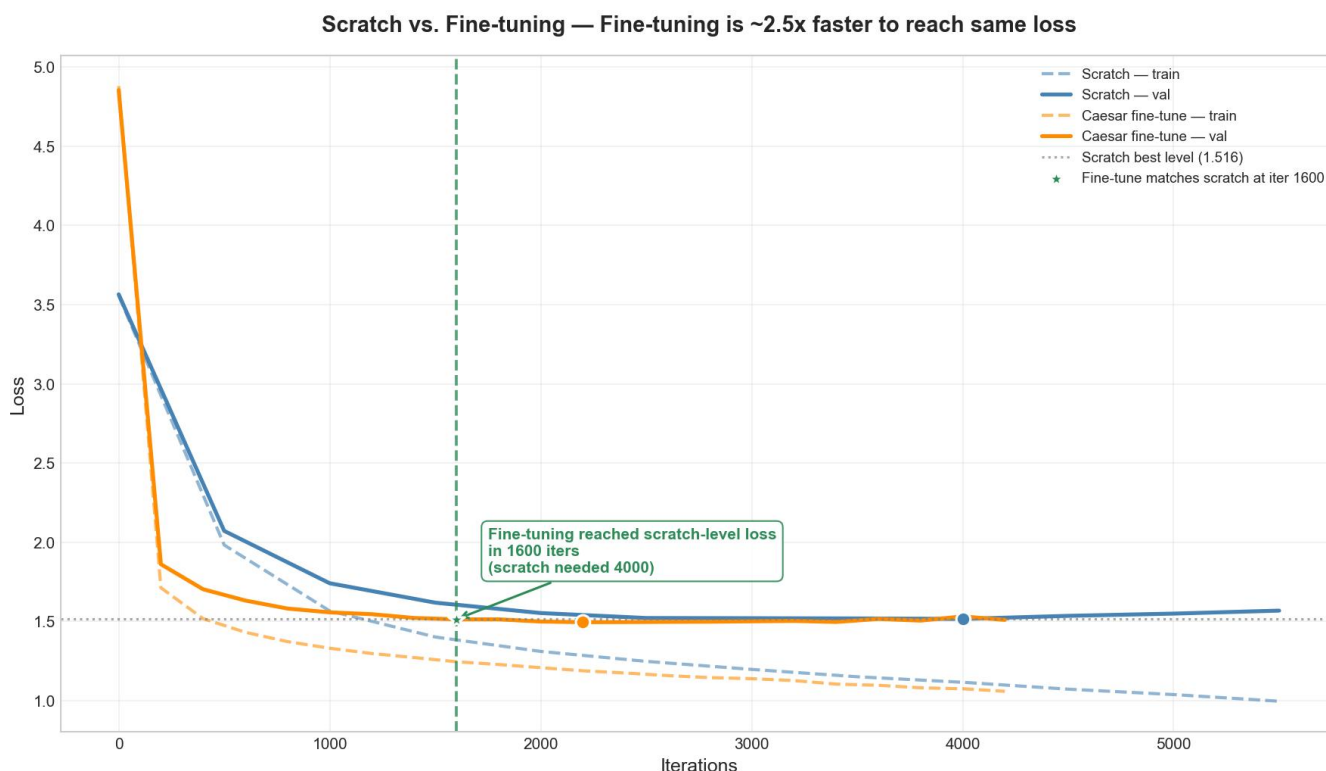


Рис. 6 Обучение двух моделей. Видно, что вторая модель дообучается до лучшего лосса первой модели за **1600** итераций, в **~2.5** раз быстрее, чем обучения с нуля.

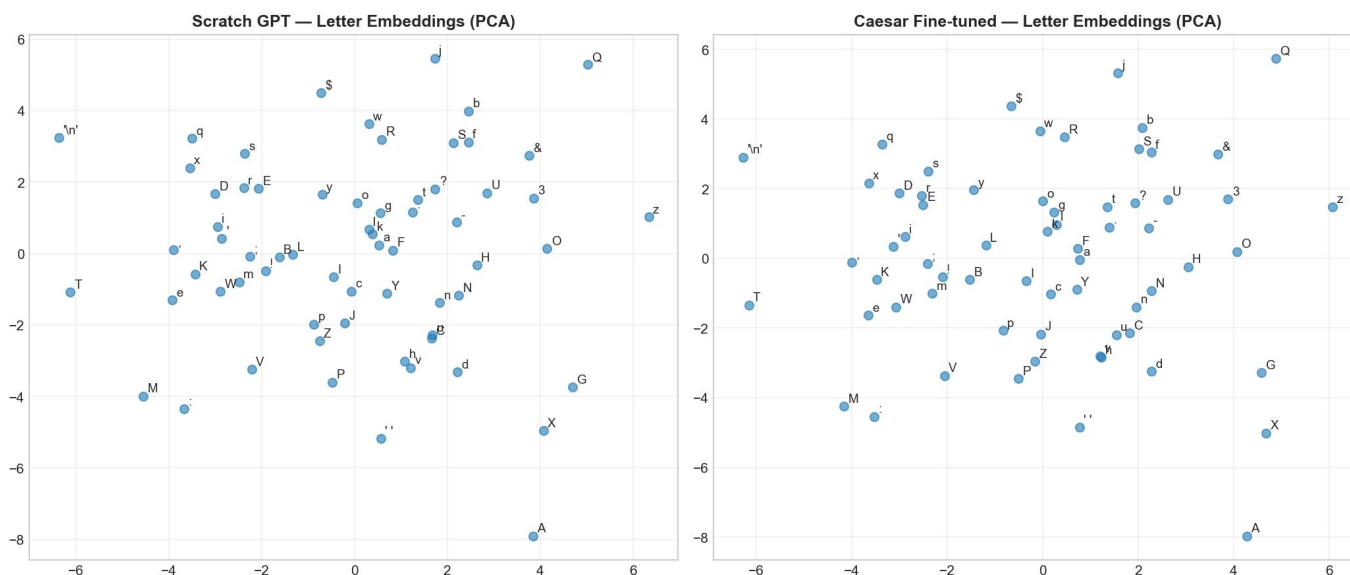


Рис. 7 Сравнение эмбедингов для букв. Тут может быть немного не понятно, но суть в том, что буква эмбединг буквы **X** в первой модели практически равен эмбедингу буквы **X+1** второй модели.

Теперь смотрим на PCA сжатие эмбедингов (рис. 7). Эмбединг для буквы ‘а’ в первой модели и для буквы ‘b’ во второй модели практически не изменились, и так для каждой буквы. То есть таблица эмбедингов практически не изменилась! Значит гипотеза опровергнута и надо искать, как же все-таки модель изменилась. Но сначала чуть больше статистики по

таблице эмбеддингов: я посчитал cosine similarity векторов эмбеддингов и составил таблицу, которая подтверждает слова выше (рис. 8).

	letter	original_letter	sim_to_same_letter	sim_to_shifted_letter
0	A	Z	0.998781	0.132631
1	B	A	0.999837	0.100593
2	C	B	0.999741	0.027479
3	D	C	0.999691	0.039048
4	E	D	0.999166	-0.093460
5	F	E	0.999839	0.020199
6	G	F	0.999656	0.070023
7	H	G	0.999701	-0.034930
8	I	H	0.999733	0.161028
9	J	I	0.999884	-0.031439
10	K	J	0.999570	-0.127768

Рис. 8 Косинусное сходство эмбеддингов букв. Видно, что эмбеддинг практически не меняется.

Хорошо, теперь сравним веса слоев моделей, а именно каждый из блоков трансформера, FeedForward и сам Attention. Я построил график того, как сильно изменились веса для каждого из слоя для каждого блока (рис. 9). Тут видно, что веса на более поздних слоях меняются сильнее, чем на начальных, при этом тенденция сохраняется от ранних блоков к поздним. Здесь в целом ничего удивительного, но суть именно в том, что сами FeedForward'ы и Attention блоки зашили в себе этот сдвиг Цезаря.

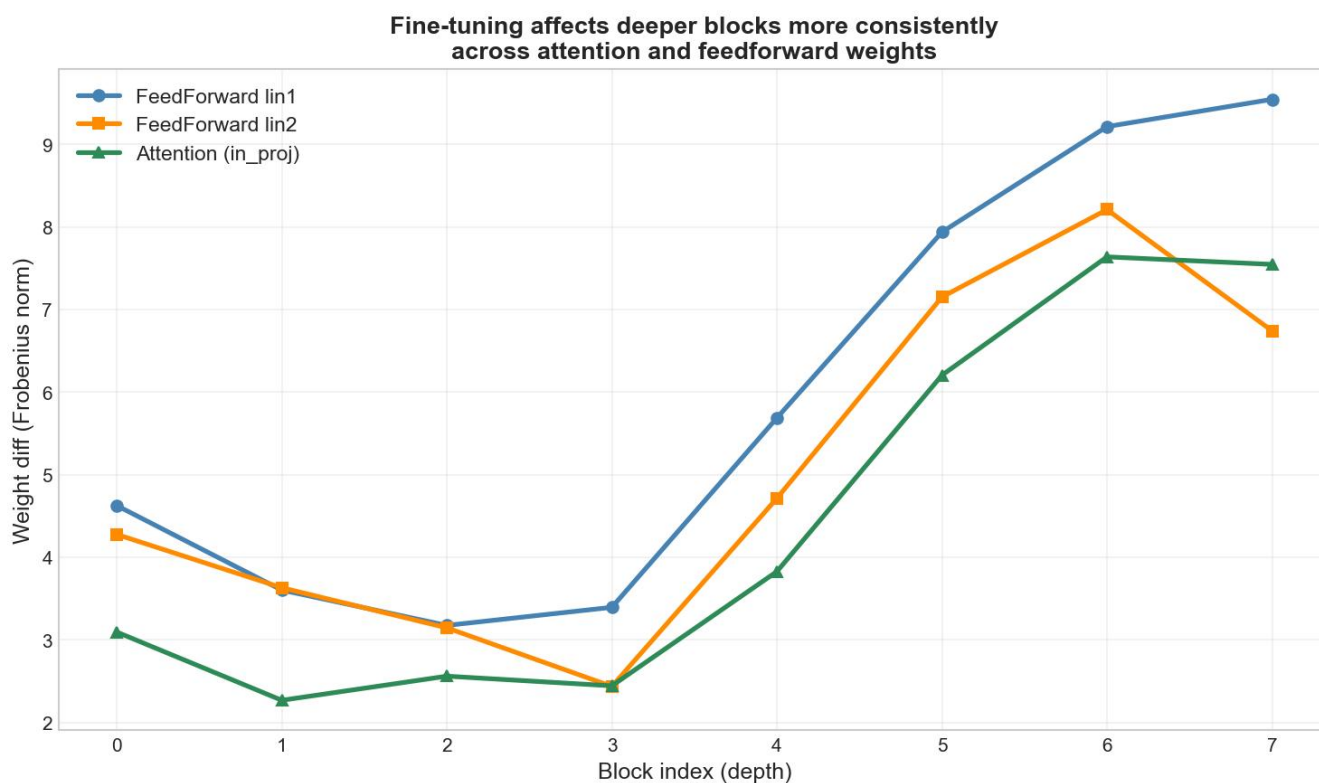


Рис. 9 Сравнение изменения весов модели в зависимости от номера блока.

Выводы

Получились весьма интересные и немного неожиданные для меня результаты! Такое «простое» шифрование проходит на более поздних слоях модели, поэтому таблица эмбедингов практически не меняется.

Дальше мне кажется весьма интересным проверить эту идею на моделях разных размеров, от маленьких, на нескольких сотен тысяч параметров, до **124M** на **GPT-mini** и больше. Думаю что для более маленьких моделей сдвиг в таблице эмбедингов более реалистичен, хотя бы потому что там меньше блоков, соответственно меньше глубины модели. Интересно также посмотреть на версию эксперимента для разных сдвигов и усреднить статистику. И в конце концов попробовать более сложные методы шифрования текста (может разворот слов??) с более сложным кодированием семантики языка и структуры предложений. А ну и конечно нормальный токенайзер использовать:D

История для тех, что дочитал:

Возможно вы знаете, что для того чтобы получить доступ к GPU на Kaggle нужно пройти верификацию личности. Для этого нужно сделать селфи, поматать головой и тд. Так значит вот что... Где-то 30 июня я попробовал пройти эту верификацию, на тот момент мне показалось что все получилось и надо просто немного подождать, пока вычислительные ресурсы появятся. Через неделю я захожу на Kaggle, вижу, ресурсов все еще нет. Попробовал пройти верификацию заново и на этот раз вышло окошко – что-то по типу *verification failed*. Я написал команде Kaggle на почту, так как мне начала закрадываться мысль о том, что меня путают с Галимом, который уже получил доступ к ресурсам. Так им и написал, что у меня есть брат близнец, который уже получил ресурсы и может быть их система верификации нас путает. На это я получил следующий ответ:

Kaggle Team (Kaggle)

Jul 8, 2026, 4:05AM PDT

Hello Khakim,

Our records show that you have multiple usernames with Kaggle: khakim, ggalim

Per the [Terms](#) agreed upon prior to account registration, please note that Kaggle allows *only one account per user*. We do not merge accounts nor transfer accomplishments from one account to another.

We recommend to continue with only your "ggalim" account.

Note this as a warning: continued creation of additional accounts can lead to disqualification to entered competitions and all of your accounts being banned.

После этого я им ответил, что у меня и вправду есть брат близнец и что его зовут Galim. И прикрепил вот это фото с указанием меня и его на картинке:



И после этого они ответили, извинились за недоразумения и выдали мне в течении 10 минут долгожданный доступ к GPU. Вот такая история про то, что нужно добиваться своего и быть готовым доказывать свою правоту :) Спасибо за внимание!